

Web VAPT Section 12 - Cross-Site Request Forgery Testing

Professional documentation for forged request proof, session trust risk, and anti-CSRF remediation.

Enterprise Security Assessment Lab

PROJECT AREA	Web Application Vulnerability Assessment and Penetration Testing
PRIMARY TARGET	DVWA running in local lab environment
TESTING ENVIRONMENT	Kali Linux attack machine with Docker-based vulnerable target
PREPARED BY	Jagriti Banerjee
DOCUMENT TYPE	Individual Web VAPT Attack-Section PDF

AUTHORIZED LAB TESTING ONLY

1. Document Purpose

This individual PDF documents the Web VAPT attack section titled 'Cross-Site Request Forgery Testing'. It is designed for the Enterprise Security Assessment Lab GitHub portfolio and describes the objective, tooling, commands, sanitized output, evidence mapping, security interpretation, remediation, and redaction requirements for this specific section.

This document is written as a public portfolio version. It does not contain live client data, production system details, unredacted credentials, session cookies, raw hash dumps, or destructive attack instructions.

Field	Details
Project Area	Web Application Vulnerability Assessment and Penetration Testing
Primary Target	DVWA running in a local lab environment
Testing Environment	Kali Linux attack machine with Docker-based vulnerable target
Testing Style	Reconnaissance -> Enumeration -> Manual validation -> Exploitation proof -> Evidence capture
Methodology Reference	OWASP Web Security Testing methodology, OWASP Top 10, and PTES-style workflow
Prepared By	Jagriti Banerjee

2. Section Overview

Item	Details
Section Number	12
Section Title	Web VAPT Section 12 - Cross-Site Request Forgery Testing
Risk / Severity Style	Medium in real-world context; higher if sensitive actions are affected
OWASP Mapping	OWASP A01 - Broken Access Control
Primary Evidence Count	2

2.1 Objectives

- Validate whether a state-changing action can be triggered by a forged request.
- Review session-based trust and missing anti-CSRF controls.
- Capture evidence without targeting real users.
- Document secure CSRF prevention controls.

2.2 Tools Used

Tool	Purpose / Use in this section
Browser	Manual verification and proof screenshots.
Burp Suite	HTTP interception and manual request testing.
DVWA CSRF module	Used as part of the controlled Web VAPT workflow.

3. Methodology and Execution Workflow

The execution workflow follows a controlled lab approach: confirm authorization and target scope, run only the tools required for the section, save outputs, validate observations manually where applicable, capture screenshots, redact sensitive values, and document findings without exaggeration.

3.1 Command Reference

```
# Sanitized CSRF proof structure
<html>
  <body>
    <form action="http://127.0.0.1:8081/vulnerabilities/csrf/" method="GET">
      <input type="hidden" name="password_new" value="<REDACTED>" />
      <input type="hidden" name="password_conf" value="<REDACTED>" />
      <input type="submit" value="Submit" />
    </form>
  </body>
</html>
```

3.2 Expected / Sanitized Output

A forged request proof was captured in the DVWA lab.
The test demonstrated that sensitive state-changing actions require anti-CSRF protection.

3.3 Validation Criteria

- The target must be the local DVWA or authorized lab environment only.
- The output must match the intended section objective.
- Evidence screenshots must clearly show the validation result or tool output.
- Any sensitive values must be blurred, cropped, removed, or replaced with placeholders.
- The section must not claim findings that are not supported by evidence.

4. Evidence Mapping

Evidence File	Evidence Type / Reason
19-csrf-forged-request-proof.png	CSRF forged request proof.
13-burp-intercepted-request.png	Burp intercepted request evidence.

5. Professional Security Analysis

CSRF relies on the browser automatically including authenticated cookies with a request. If an application does not require a unique anti-CSRF token, a malicious page may cause unwanted actions.

CSRF impact depends on the sensitivity of the affected function. Password change, email change, financial actions, or admin actions are higher risk.

Professional CSRF reporting should include affected endpoint, request method, missing token, session cookie behaviour, and proof of action.

5.1 Why This Section Matters

This section matters because professional VAPT is not only about running a tool. It requires a clear objective, controlled execution, evidence capture, correct interpretation, and practical remediation. For recruiters and reviewers, this PDF shows that the work was structured, documented, and aligned with real security methodology.

5.2 Common Mistakes to Avoid

- Uploading raw outputs that expose cookies, hashes, paths, or credentials.
- Claiming production-level impact without production context.
- Reporting scanner output as a confirmed vulnerability without validation.
- Using unclear screenshots that do not show the relevant result.
- Mixing unrelated phases in one evidence file without explanation.

6. Risk Analysis

- Unauthorized account changes.
- Password or email modification.
- Forced state changes.
- Abuse of authenticated user context.
- Administrator action abuse if admin functions are affected.

6.1 Real-World Impact Statement

The real-world impact depends on the affected asset, authentication context, data sensitivity, privilege level, exposed functionality, and compensating controls. Because this project uses DVWA in a lab, severity is described as a real-world context estimate rather than a formal client CVSS score.

7. Remediation and Hardening Guidance

- Use unpredictable anti-CSRF tokens.
- Validate tokens server-side for every state-changing request.
- Set SameSite cookie attributes.
- Validate Origin and Referer headers as defense-in-depth.
- Require re-authentication for critical actions.
- Avoid using GET for state-changing operations.

7.1 Verification After Remediation

- 1 Re-run the original test case in the lab or development environment.
- 2 Confirm that the previous vulnerable behaviour no longer appears.
- 3 Check server logs for blocked or rejected attempts.
- 4 Capture new evidence showing the fix or defensive control.

- 5 Update the report status from validated/open to fixed/retested if applicable.

8. GitHub Redaction and Publishing Checklist

Cookies, hashes, credentials, session IDs, shell paths, database output, and private details must be redacted before public upload.

Item	Action Before Upload
PHPSESSID / Cookies	Blur or replace with <REDACTED_COOKIE>.
Password hashes	Blur or replace with <REDACTED_HASH>.
Cracked passwords	Do not publish; replace with <REDACTED_PASSWORD>.
Database dumps	Crop, blur, or summarize instead of uploading raw data.
Webshell paths/files	Do not upload shell source files; redact paths if needed.
Local usernames/paths	Blur if private or unnecessary.
Tool raw outputs	Upload only sanitized outputs or summary documents.

8.1 Safe Git Commands Before Push

```
git status
git diff --cached
git diff --cached | grep -iE "password|secret|token|cookie|PHPSESSID|hash|credential"
```

9. Conclusion

This document completes the individual Web VAPT PDF for Section 12. It documents the section objective, execution workflow, evidence mapping, professional interpretation, risk, remediation, and GitHub safety checks. The content is aligned to the actual Enterprise Security Assessment Lab Web VAPT evidence set and avoids unsupported production claims.

Legal and ethical notice: This documentation is for authorized lab testing only. Do not apply these techniques to public or third-party systems without explicit written permission.